INTERNATIONALE HOCHSCHULE

Projektphase 1.2

# Erarbeitungs-/Reflexionsphase

Book Exchange App

Februar 2026

Marie Groß, Matrikelnummer: IU14106516

# Inhaltsverzeichnis

1. Teil I: Operatives Datenmodell
2. Teil II: Prozesslogik und Trigger
3. Teil III: Funktionale Validierung
4. Teil IV: Data Mart und Analysen

# Teil I: Operatives Datenmodell

# Entität: Member

Speichert registrierte Mitglieder der Buchtausch-App mit Authentifizierungs- und Profildaten.

## SQL STATEMENT

```sql
CREATE TABLE member (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(80) NOT NULL,
    last_name VARCHAR(80) NOT NULL,
    email_address VARCHAR(254) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    phone_number VARCHAR(30),
    registration_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    profile_picture_url VARCHAR(1024),
    about_me  TEXT,
    status ENUM('active', 'inactive', 'suspended', 'deleted') NOT NULL DEFAULT 'active',
    role ENUM('member', 'admin') NOT NULL DEFAULT 'member',

    CONSTRAINT chk_member_email CHECK (email_address LIKE '%@%.%'),
    CONSTRAINT uq_member_email UNIQUE (email_address),

    INDEX index_member_status (status)

) ENGINE=INNODB COMMENT='Registered members of the book exchange app';

INSERT INTO member (first_name, last_name, email_address, password_hash, phone_number,
profile_picture_url, about_me, status, role) VALUES
('Anna', 'Müller', 'anna.mueller@example.com', '$2a$12$KIXxP9H7wZ8vN2qL5RtMveP7Q8wH3N9xYzE4kF6mR7sT8uV9wX0yK',
'+49 176 12345678', 'https://example.com/profiles/anna_m.jpg', 'Passionate reader of classic literature and philosophy.
Always looking for book recommendations!', 'active', 'member')
```

bookexchange.member: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 first_name | 3 last_name | 4 email_address | 5 password_hash | 6 phone_number | 7 registration_date | 8 profile_picture_url | 9 about_me | 10 status | 11 role |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Anna | Müller | anna.mueller@example.com | $2a$12$KIXxP9H7wZ8vN2qL5RtMveP7Q8wH3N9xYzE4k... | +49 176 12345678 | 2025-11-23 12:19:48 | https://example.com/profiles/anna_m.jpg | Passionate reader of classic literature and philosophy. Al... | active | member |
| 2 | 2 | Ben | Schmidt | ben.schmidt@example.com | $2a$12$LJYyQ0I8xA9wO3rM6StNwfQ8R9xI4O0zAF5lG7... | (NULL) | 2025-11-23 12:19:48 | https://example.com/profiles/ben_s.jpg | Science fiction enthusiast. Love trading sci-fi and fantasy... | active | member |
| 3 | 3 | Carla | Schneider | carla.schneider@example.com | $2a$12$MKZzR1J9yB0xP4sN7TuOxgR9S0yJ5P1aB6mH8... | +49 151 98765432 | 2025-11-23 12:19:48 | https://example.com/profiles/carla_sch.jpg | Platform administrator. Here to help with any issues! | active | admin |
| 4 | 4 | David | Fischer | david.fischer@example.com | $2a$12$NLAaS2K0zC1yQ5tO8UvPyhS0T1zK6Q2bC7nI9p... | +49 160 11223344 | 2025-11-23 12:19:48 | (NULL) | Looking for Spanish and Latin American literature. Open t... | active | member |
| 5 | 5 | Eva | Weber | eva.weber@example.com | $2a$12$OMBbT3L1aD2zR6uP9VwQziT1U2aL7R3cD8oJ0q... | (NULL) | 2025-11-23 12:19:48 | (NULL) | Taking a break from reading. Will be back soon! | inactive | member |
| 6 | 6 | Felix | Meyer | felix.meyer@example.com | $2a$12$PNCcU4M2bE3aS7vQ0WxRajU2V3bM8S4dE9pK1... | +49 171 55667788 | 2025-11-23 12:19:48 | https://example.com/profiles/felix_m.jpg | (NULL) | active | member |
| 7 | 7 | Greta | Wagner | greta.wagner@example.com | $2a$12$QODdV5N3cF4bT8wR1XySbkV3W4cN9T5eF0qL2... | +49 173 44332211 | 2025-11-23 12:19:48 | (NULL) | (NULL) | active | member |
| 8 | 8 | Hans | Becker | hans.becker@example.com | $2a$12$RPEeW6O4dG5cU9xS2YzTdW4X5dO0U6fG1rM3... | (NULL) | 2025-11-23 12:19:48 | (NULL) | (NULL) | suspended | member |
| 9 | 9 | Ida | Hoffmann | ida.hoffmann@example.com | $2a$12$SQFfX7P5eH6dV0yT3ZaUdmX5Y6eP1V7gH2sN4u... | +49 173 99887766 | 2025-11-23 12:19:48 | https://example.com/profiles/ida_h.jpg | Harry Potter fan! Also love fantasy and young adult ficti... | active | member |
| 10 | 10 | Jan | Schulz | jan.schulz@example.com | $2a$12$TRGgY8Q6fl7eW1zU4AbVenY6Z7fQ2W8hI3tO5... | (NULL) | 2025-11-23 12:19:48 | (NULL) | (NULL) | active | member |

## HIGHLIGHTS

- **Status Management:** ENUM definiert Benutzerzustände (z. B. active, suspended).
- **Datenvalidierung:** CHECK Constraint erzwingt Mindestformat für E-Mail-Adressen (‘@‘ und einen ‘.‘).
- **Performance:** Index auf email_address (Login) für Authentifizierung.

# Entität: Language

Enthält verfügbare Sprachen für Bücher zur Vermeidung von Redundanz.

## SQL STATEMENT

```sql
CREATE TABLE language (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    iso_code VARCHAR(5) NOT NULL,
    name VARCHAR(80) NOT NULL,

    CONSTRAINT uq_language_iso  UNIQUE (iso_code),
    CONSTRAINT uq_language_name UNIQUE (name)

) ENGINE=InnoDB COMMENT='Available languages for books (ISO 639-1)';

INSERT INTO language (iso_code, name) VALUES
('de', 'Deutsch')
```

bookexchange.language: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 iso_code | 3 name |
|---|---|---|---|
| 1 | 1 | de | Deutsch |
| 2 | 2 | en | English |
| 3 | 3 | fr | Français |
| 4 | 4 | es | Español |
| 5 | 5 | it | Italiano |
| 6 | 6 | pt | Português |
| 7 | 7 | nl | Nederlands |
| 8 | 8 | pl | Polski |
| 9 | 9 | tr | Türkçe |
| 10 | 10 | ja | 日本語 |

## HIGHLIGHTS

- **Internationalisierung:** Unterstützung durch ISO-Codes ( z. B. ‘de‘, ‘en‘, ‘fr‘).
- **Lookup-Table:** Zentrale Verwaltung erlaubter Sprachen für das System.

# Entität: Publisher

Speichert Informationen über Verlage und deren Standorte.

## SQL STATEMENT

```sql
CREATE TABLE publisher (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(80) NOT NULL,
    headquarter VARCHAR(80),

    CONSTRAINT uq_publisher_name UNIQUE (name)

) ENGINE=InnoDB COMMENT='Publishers';

INSERT INTO publisher (name, headquarter) VALUES
('Suhrkamp Verlag', 'Berlin')
```

bookexchange.publisher: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 name | 3 headquarter |
|---|---|---|---|
| 1 | 1 | Suhrkamp Verlag | Berlin |
| 2 | 2 | Penguin Random House | New York |
| 3 | 3 | Hanser Verlag | (NULL) |
| 4 | 4 | Rowohlt Verlag | Hamburg |
| 5 | 5 | Gallimard | Paris |
| 6 | 6 | Fischer Verlag | Frankfurt am Main |
| 7 | 7 | HarperCollins | New York |
| 8 | 8 | Kiepenheuer & Witsch | (NULL) |
| 9 | 9 | Diogenes Verlag | Zürich |
| 10 | 10 | C.H. Beck | München |

## HIGHLIGHTS

- **Normalisierung (3. NF):** Auslagerung von Verlagsnamen zur Vermeidung von Redundanz.
- **Datenqualität:** UNIQUE Constraint verhindert Duplikate (z. B. doppelte Einträge desselben Verlags).

# Entität: Author

Enthält Autoren von Büchern mit biografischen Informationen.

## SQL STATEMENT

```sql
CREATE TABLE author (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(80) NOT NULL,
    last_name VARCHAR(80) NOT NULL,
    bio TEXT,

    INDEX idx_author_name (last_name, first_name)

) ENGINE=InnoDB COMMENT='Authors of books';

INSERT INTO author (first_name, last_name, bio) VALUES
('Franz', 'Kafka', 'Bedeutender deutschsprachiger Schriftsteller des 20. Jahrhunderts.')
```

bookexchange.author: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 first_name | 3 last_name | 4 bio |
|---|---|---|---|---|
| 1 | 1 | Franz | Kafka | Bedeutender deutschsprachiger Schriftsteller des 20. Jah... |
| 2 | 2 | Haruki | Murakami | Japanischer Autor von Romanen und Kurzgeschichten, b... |
| 3 | 3 | Margaret | Atwood | Kanadische Dichterin, Romanautorin und Essayistin. |
| 4 | 4 | Carlos | Ruiz Zafón | (NULL) |
| 5 | 5 | Elena | Ferrante | (NULL) |
| 6 | 6 | Michel | Houellebecq | (NULL) |
| 7 | 7 | J.K. | Rowling | Britische Schriftstellerin, Autorin der Harry-Potter-Serie. |
| 8 | 8 | George | Orwell | Englischer Schriftsteller, Essayist und Journalist, bekannt... |
| 9 | 9 | Yuval Noah | Harari | (NULL) |
| 10 | 10 | Juli | Zeh | (NULL) |

## HIGHLIGHTS

- **Performance:** Zusammengesetzter Index (last_name, first_name) für alphabetische Sortierung.
- **Inhalt:** Speicherung von Zusatzinfos (Bio) direkt am Autoren-Objekt.

# Entität: Genre

Normalisierte Tabelle für Buchgenres und deren Beschreibungen.

## SQL STATEMENT

```sql
CREATE TABLE genre (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(80) NOT NULL,
    description TEXT,

    CONSTRAINT uq_genre_name UNIQUE (name)

) ENGINE=InnoDB COMMENT='Book genres';

INSERT INTO genre (name, description) VALUES
('Roman', 'Epische Großform der Literatur.')
```

bookexchange.genre: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 name | 3 description |
|---|---|---|---|
| 1 | 1 | Roman | Epische Großform der Literatur. |
| 2 | 2 | Science-Fiction | Literatur, die sich mit fiktiven zukünftigen Entwicklungen ... |
| 3 | 3 | Krimi | (NULL) |
| 4 | 4 | Fantasy | Genre der Phantastik mit magischen oder übernatürlichen... |
| 5 | 5 | Sachbuch | Vermittelt Fakten und reales Wissen zu einem bestimmten... |
| 6 | 6 | Biografie | (NULL) |
| 7 | 7 | Dystopie | Erzählung, die in einer düsteren, unerwünschten Zukunft... |
| 8 | 8 | Thriller | Erzeugt Spannung und Nervenkitzel, oft mit einer drohen... |
| 9 | 9 | Historischer Roman | Roman, dessen Handlung in einer vergangenen Epoche s... |
| 10 | 10 | Gegenwartsliteratur | (NULL) |

## HIGHLIGHTS

- **Klassifizierung:** Eindeutige Kategorienamen durch UNIQUE Constraint.
- **Suche:** Index auf Namen ermöglicht schnelle Filterung im Frontend.

# Entität: Book

Repräsentiert Buch-Metadaten (Werke/Editionen), nicht physische Exemplare.

## SQL STATEMENT

```sql
CREATE TABLE book (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    publisher_id BIGINT,
    language_id BIGINT,
    title VARCHAR(255) NOT NULL,
    subtitle VARCHAR(255),
    publication_year YEAR,
    isbn VARCHAR(20),
    pages INT,
    cover_url VARCHAR(1024),
    description TEXT,

    FOREIGN KEY (publisher_id) REFERENCES publisher(id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (language_id) REFERENCES language(id)
        ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT chk_book_isbn CHECK (isbn IS NULL OR LENGTH(isbn) IN (10, 13)),
    CONSTRAINT chk_book_pages CHECK (pages IS NULL OR pages > 0),

    CONSTRAINT uq_book_isbn UNIQUE (isbn),

    FULLTEXT INDEX idx_book_title (title, subtitle),
    INDEX idx_book_publisher (publisher_id),
    INDEX idx_book_language (language_id)

) ENGINE=InnoDB COMMENT='Books (works/editions) - not physical copies';

INSERT INTO book (publisher_id, language_id, title, subtitle, publication_year,
isbn, pages, description) VALUES
(1, 1, 'Der Prozess', NULL, 1925, '9783518188000', 255,
'Ein Romanfragment von Franz Kafka, posthum veröffentlicht.')
```

## HIGHLIGHTS

- **Suchoptimierung:** FULLTEXT INDEX für performante Titelsuche.
- **Datenqualität:** CHECK validiert ISBN-Länge (10/13).
- **Metadaten:** Trennung vom physischen Exemplar.

bookexchange.book: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 publisher_id | 3 language_id | 4 title | 5 subtitle | 6 publication_year | 7 isbn | 8 pages | 9 cover_url | 10 description |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | Der Prozess | (NULL) | 1925 | 9783518188000 | 255 | (NULL) | Ein Romanfragment von Franz Kafka, posthum veröffentl... |
| 2 | 2 | 8 | 1 | 1Q84 | Buch 1 & 2 | 2010 | 9783832161486 | 1.040 | (NULL) | Eine surreale Reise in eine Parallelwelt im Jahr 1984. |
| 3 | 3 | 2 | 2 | The Handmaids Tale | (NULL) | 1985 | 9780385490818 | 311 | (NULL) | Ein dystopischer Roman über eine totalitäre Gesellschaft. |
| 4 | 4 | 6 | 4 | La Sombra del Viento | (NULL) | 2001 | 9788408043640 | 565 | (NULL) | Ein junger Mann entdeckt ein geheimnisvolles Buch auf d... |
| 5 | 5 | 1 | 5 | Lamica geniale | (NULL) | 2011 | 9788866320326 | 450 | (NULL) | Der erste Band der Neapolitanischen Saga über die Freun... |
| 6 | 6 | 5 | 3 | Sérotonine | (NULL) | 2019 | 9782072815528 | 347 | (NULL) | Ein Roman über Depression Landwirtschaft und die mode... |
| 7 | 7 | 2 | 2 | Harry Potter and the Philosophers Stone | (NULL) | 1997 | 9780747532699 | 223 | (NULL) | Der erste Band der weltberühmten Zauberer-Saga. |
| 8 | 8 | 2 | 2 | Nineteen Eighty-Four | 1984 | 1949 | 9780451524935 | 328 | (NULL) | Ein ikonischer dystopischer Roman über Überwachung un... |
| 9 | 9 | 10 | 1 | Eine kurze Geschichte der Menschheit | (NULL) | 2013 | 9783570552698 | 528 | (NULL) | Ein Sachbuch über die Geschichte des Homo sapiens. |
| 10 | 10 | 1 | 1 | Unterleuten | (NULL) | 2016 | 9783630874876 | 672 | (NULL) | Ein Gesellschaftsroman über einen Konflikt in einem brand... |

# Entität: Book_author

Verknüpfungstabelle für die n:m-Beziehung zwischen Büchern und Autoren.

## SQL STATEMENT

```sql
CREATE TABLE book_author (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    book_id BIGINT NOT NULL,
    author_id BIGINT NOT NULL,

    FOREIGN KEY (book_id) REFERENCES book(id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (author_id) REFERENCES author(id) ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT uq_book_author UNIQUE (book_id, author_id),

    INDEX idx_book_author_reverse (author_id, book_id)

) ENGINE=InnoDB COMMENT='Link table books-authors (N:M)';

INSERT INTO book_author (book_id, author_id) VALUES
(1, 1),  -- Der Prozess -> Franz Kafka
```

bookexchange.book_author: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 book_id | 3 author_id |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 2 |
| 3 | 3 | 3 | 3 |
| 4 | 4 | 4 | 4 |
| 5 | 5 | 5 | 5 |
| 6 | 6 | 6 | 6 |
| 7 | 7 | 7 | 7 |
| 8 | 8 | 8 | 8 |
| 9 | 9 | 9 | 9 |
| 10 | 10 | 10 | 10 |

## HIGHLIGHTS

- **n:m Auflösung:** Ein Buch kann mehrere Autoren haben.
- **Integrität:** UNIQUE verhindert doppelte Zuweisungen.
- **Wartung:** CASCADE löscht Link bei Buchlöschung.

# Entität: Book_genre

Verknüpfungstabelle für die n:m-Beziehung zwischen Büchern und Genres.

## SQL STATEMENT

```sql
CREATE TABLE book_genre (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    book_id BIGINT NOT NULL,
    genre_id BIGINT NOT NULL,

    FOREIGN KEY (book_id) REFERENCES book(id)  ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (genre_id) REFERENCES genre(id) ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT uq_book_genre UNIQUE (book_id, genre_id),

    INDEX idx_book_genre_reverse (genre_id, book_id)

) ENGINE=InnoDB COMMENT='Link table books-genres (N:M)';

INSERT INTO book_genre (book_id, genre_id) VALUES
(1, 1),  -- Der Prozess -> Roman
```

bookexchange.book_genre: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 book_id | 3 genre_id |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 1 |
| 3 | 3 | 3 | 7 |
| 4 | 4 | 4 | 9 |
| 5 | 5 | 5 | 1 |
| 6 | 6 | 6 | 10 |
| 7 | 7 | 7 | 4 |
| 8 | 8 | 8 | 7 |
| 9 | 9 | 9 | 5 |
| 10 | 10 | 10 | 10 |

## HIGHLIGHTS

- **Flexibilität:** Bücher können mehreren Genres angehören.
- **Reverse Index:** Ermöglicht schnelles Finden aller Bücher eines Genres.

# Entität: Book_copy

Repräsentiert einzelne physische Buchexemplare im Besitz von Mitgliedern.

## SQL STATEMENT

```sql
CREATE TABLE book_copy (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    book_id BIGINT NOT NULL,
    owner_id BIGINT NOT NULL,
    condition_code ENUM('new', 'like_new', 'very_good', 'good', 'acceptable', 'poor') NOT NULL,
    condition_description TEXT,
    acquisition_date DATE,
    notes TEXT,
    status ENUM('available', 'on_loan', 'unavailable') NOT NULL DEFAULT 'available',

    FOREIGN KEY (book_id) REFERENCES book(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (owner_id) REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,

    INDEX idx_copy_book (book_id),
    INDEX idx_copy_owner (owner_id),
    INDEX idx_copy_status (status),
    INDEX idx_copy_condition (condition_code)

) ENGINE=InnoDB COMMENT='Physical book copies owned by members';

INSERT INTO book_copy (book_id, owner_id, condition_code, condition_description, acquisition_date, notes, status) VALUES
(1, 1, 'good', 'Some dog-eared pages on chapters 3 and 7. Minor wear on spine. Overall still in good reading condition.',
'2020-05-15', 'Purchased at a local bookstore. Has my name written inside the cover.', 'available')
```

bookexchange.book_copy: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 book_id | 3 owner_id | 4 condition_code | 5 condition_description | 6 acquisition_date | 7 notes | 8 status |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | good | Some dog-eared pages on chapters 3 and 7. Minor wear ... | 2020-05-15 | Purchased at a local bookstore. Has my name written insi... | available |
| 2 | 2 | 2 | 2 | like_new | Only read once, no visible signs of use. Pages are crisp a... | (NULL) | Gift from a friend. Happy to lend it out! | available |
| 3 | 3 | 3 | 3 | very_good | Minimal wear, slight discoloration on the edges of some p... | 2019-11-20 | (NULL) | available |
| 4 | 4 | 4 | 4 | acceptable | Visible creases on spine, some pages have highlighting an... | (NULL) | (NULL) | available |
| 5 | 5 | 5 | 6 | new | (NULL) | 2024-12-01 | (NULL) | available |
| 6 | 6 | 6 | 7 | good | Slight yellowing on page edges due to age. Cover has mi... | (NULL) | (NULL) | available |
| 7 | 7 | 7 | 9 | very_good | Hardcover edition in excellent condition. Dust jacket inclu... | 2021-08-10 | Part of my collection. First edition! | available |
| 8 | 8 | 8 | 10 | poor | Several pages are loose but still attached. Cover is heavil... | 2015-03-22 | Found at a flea market. Still readable despite condition. | available |
| 9 | 9 | 9 | 1 | like_new | Almost perfect condition. Read very carefully. No crease... | 2023-06-18 | Currently lent to a friend. Expected back in 2 weeks. | on_loan |
| 10 | 10 | 10 | 2 | good | Normal wear from regular use. A few coffee stains on co... | (NULL) | (NULL) | unavailable |

## HIGHLIGHTS

- **Inventar:** Physisches Exemplar vs. abstraktes Buch.
- **Zustand:** ENUM für standardisierte Qualität.
- **Besitz:** Bindung an User über owner_id.

# Entität: Location

Speichert geografische Standorte (Adressen), an denen Bücher abgeholt werden können.

## SQL STATEMENT

```sql
CREATE TABLE location (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    member_id BIGINT NOT NULL,
    street VARCHAR(120) NOT NULL,
    house_number VARCHAR(20) NOT NULL,
    postal_code VARCHAR(10) NOT NULL,
    city VARCHAR(120) NOT NULL,
    country_code CHAR(2) DEFAULT 'DE',
    coordinates POINT NOT NULL,
    label ENUM('home', 'work', 'pickup', 'other') NOT NULL DEFAULT 'home',

    FOREIGN KEY (member_id) REFERENCES member(id) ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT chk_location_srid CHECK (ST_SRID(coordinates) = 4326),

    INDEX idx_location_member (member_id),
    INDEX idx_location_city (city),
    INDEX idx_location_postal_code (postal_code),
    SPATIAL INDEX idx_location_coordinates (coordinates)

) ENGINE=InnoDB COMMENT='Addresses and pickup locations for members';

INSERT INTO location (member_id, street, house_number, postal_code, city, country_code, coordinates, label) VALUES
(1, 'Hauptstraße', '10', '10115', 'Berlin', 'DE', ST_PointFromText('POINT(13.4050 52.5200)', 4326), 'home')
```

bookexchange.location: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 member_id | 3 street | 4 house_number | 5 postal_code | 6 city | 7 country_code | 8 coordinates | 9 label |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | Hauptstraße | 10 | 10115 | Berlin | DE | æ□ □□ □Âõ(\Ï*@Ãõ(\ÞJ@ | home |
| 2 | 2 | 2 | Marktplatz | 5a | 20095 | Hamburg | DE | æ□ □□ □□□?ÆÚ#@-HÞqŠÆJ@ | home |
| 3 | 3 | 3 | Büroallee | 120 | 80331 | München | DE | æ□ □□ D·4çÛ)'@EØBõJ□H@ | work |
| 4 | 4 | 4 | Sonnenweg | 1 | 10557 | Berlin | DE | æ□ □□ □4□6<½*@&S□£ÞJ@ | home |
| 5 | 5 | 5 | Alter Weg | 22 | 50667 | Köln | DE | æ□ □□ Ü×sFÔ□@ xI@ | home |
| 6 | 6 | 6 | Bahnhofstraße | 7 | 60329 | Frankfurt am Main | DE | æ□ □□ û\mÅþR!@iø1æe@I@ | pickup |
| 7 | 7 | 7 | Rue de la Paix | 15 | 75002 | Paris | FR | æ□ □□ ,Õšæ§ @□□□Ã1oH@ | home |
| 8 | 8 | 8 | Nebengasse | 3 | 10115 | Berlin | DE | æ□ □□ Zd;ßOÍ*@¦›Ã ªBJ@ | home |
| 9 | 9 | 9 | Elbchaussee | 100 | 22605 | Hamburg | DE | æ□ □□ zÇ):È#@@¶ÞiÙÅJ@ | home |
| 10 | 10 | 10 | Turmstraße | 30 | 10559 | Berlin | DE | æ□ □□ ¼çû©±*@û\mÅþBJ@ | work |

## HIGHLIGHTS

- **Spatial Data:** Speicherung als POINT ermöglicht performante Radius-Suche (“Bücher in meiner Nähe“).
- **Referenzielle Integrität:** ON DELETE CASCADE löscht verwaiste Adressen automatisch.
- **Standardisierung:** SRID 4326 erzwingt das GPS Standardformat (WGS 84).

# Entität: Availability

Definiert Verfügbarkeitszeiträume und Ausleihbedingungen für Buchexemplare.

## SQL STATEMENT

```sql
CREATE TABLE availability (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    copy_id BIGINT NOT NULL,
    location_id BIGINT NOT NULL,
    available_from DATE NOT NULL,
    available_to DATE NOT NULL,
    max_duration_days SMALLINT DEFAULT 14,
    shipping_possible BOOLEAN NOT NULL DEFAULT false,
    status ENUM('active', 'paused', 'archived') NOT NULL DEFAULT 'active',

    FOREIGN KEY (copy_id) REFERENCES book_copy(id) ON DELETE CASCADE  ON UPDATE CASCADE,
    FOREIGN KEY (location_id) REFERENCES location(id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT chk_availability_dates CHECK (available_from <= available_to),
    CONSTRAINT chk_availability_duration CHECK (max_duration_days > 0),

    INDEX idx_availability_copy (copy_id),
    INDEX idx_availability_location (location_id),
    INDEX idx_availability_status (status),
    INDEX idx_availability_dates (available_from, available_to)

) ENGINE=InnoDB COMMENT='Availability periods for book copies';

INSERT INTO availability (copy_id, location_id, available_from, available_to, max_duration_days,
shipping_possible, status) VALUES
(1, 1, '2024-01-01', '2025-12-31', 21, FALSE, 'active')
```

## HIGHLIGHTS

- **Logik:** CHECK verhindert unmögliche Zeiträume.
- **Flexibilität:** Buch an verschiedenen Orten verfügbar.
- **Planung:** Zeitfenster statt permanenter Verfügbarkeit.

bookexchange.availability: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 copy_id | 3 location_id | 4 available_from | 5 available_to | 6 max_duration_days | 7 shipping_possible | 8 status |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2024-01-01 | 2025-12-31 | 21 | 0 | active |
| 2 | 2 | 2 | 2 | 2024-01-01 | 2025-12-31 | 14 | 1 | active |
| 3 | 3 | 3 | 3 | 2024-05-01 | 2025-05-01 | 10 | 0 | active |
| 4 | 4 | 4 | 4 | 2024-01-01 | 2025-12-31 | 30 | 1 | active |
| 5 | 5 | 5 | 6 | 2024-01-01 | 2025-12-31 | 14 | 0 | active |
| 6 | 6 | 6 | 7 | 2024-01-01 | 2025-12-31 | 14 | 1 | paused |
| 7 | 7 | 7 | 9 | 2024-01-01 | 2025-12-31 | 14 | 1 | active |
| 8 | 8 | 8 | 10 | 2024-01-01 | 2025-12-31 | 7 | 0 | active |
| 9 | 9 | 9 | 1 | 2024-01-01 | 2025-12-31 | 14 | 0 | archived |
| 10 | 10 | 10 | 2 | 2024-01-01 | 2025-12-31 | 14 | 1 | active |

# Entität: Loan

Verfolgt den kompletten Ausleihvorgang von der Anfrage bis zur Rückgabe.

## SQL STATEMENT

```sql
CREATE TABLE loan (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    availability_id BIGINT NOT NULL,
    lender_id BIGINT NOT NULL,
    borrower_id BIGINT NOT NULL,
    request_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    start_date DATE,
    planned_end_date DATE,
    actual_end_date DATE,
    status ENUM('requested', 'accepted', 'active', 'returned', 'cancelled', 'overdue') NOT NULL DEFAULT 'requested',
    cancellation_date DATE,
    cancellation_reason TEXT,

    FOREIGN KEY (availability_id) REFERENCES availability(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (lender_id) REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (borrower_id) REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT chk_loan_start_date CHECK (start_date <= planned_end_date OR planned_end_date IS NULL),
    CONSTRAINT chk_loan_actual_date CHECK (actual_end_date >= start_date OR actual_end_date IS NULL),

    INDEX idx_loan_availability (availability_id),
    INDEX idx_loan_lender (lender_id),
    INDEX idx_loan_borrower (borrower_id),
    INDEX idx_loan_status (status),
    INDEX idx_loan_dates (start_date, planned_end_date)

) ENGINE=InnoDB COMMENT='Loan transactions - full lifecycle from request to return';

INSERT INTO loan (availability_id, lender_id, borrower_id, request_date, start_date, planned_end_date,
actual_end_date, status, cancellation_date, cancellation_reason) VALUES
(1, 1, 2, '2025-09-25 14:30:00', '2025-10-01', '2025-10-22', '2025-10-20', 'returned', NULL, NULL)
```

bookexchange.loan: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 availability_id | 3 lender_id | 4 borrower_id | 5 request_date | 6 start_date | 7 planned_end_date | 8 actual_end_date | 9 status | 10 cancellation_d... | 11 cancellation_reason |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2 | 2024-09-25 14:30:00 | 2024-10-01 | 2024-10-22 | 2024-10-20 | returned | (NULL) | (NULL) |
| 2 | 2 | 2 | 2 | 3 | 2024-09-28 09:15:00 | 2024-10-05 | 2024-10-19 | 2024-10-18 | returned | (NULL) | (NULL) |
| 3 | 3 | 3 | 3 | 4 | 2024-10-20 16:45:00 | 2024-11-01 | 2024-11-11 | 2024-11-13 | returned | (NULL) | (NULL) |
| 4 | 4 | 4 | 4 | 6 | 2024-10-28 11:20:00 | 2024-11-05 | 2024-12-05 | 2024-12-01 | returned | (NULL) | (NULL) |
| 5 | 5 | 5 | 6 | 7 | 2024-11-01 08:00:00 | 2024-11-10 | 2024-11-24 | 2024-11-23 | returned | (NULL) | (NULL) |
| 6 | 6 | 6 | 7 | 9 | 2025-11-05 10:30:00 | 2025-11-10 | 2025-11-24 | (NULL) | active | (NULL) | (NULL) |
| 7 | 7 | 7 | 9 | 10 | 2025-11-18 13:45:00 | (NULL) | (NULL) | (NULL) | requested | (NULL) | (NULL) |
| 8 | 8 | 8 | 10 | 1 | 2025-10-15 15:20:00 | 2025-10-20 | 2025-10-27 | (NULL) | overdue | (NULL) | (NULL) |
| 9 | 9 | 9 | 1 | 4 | 2025-10-25 12:00:00 | 2025-11-01 | 2025-11-15 | (NULL) | active | (NULL) | (NULL) |
| 10 | 10 | 10 | 2 | 1 | 2025-11-12 17:30:00 | (NULL) | (NULL) | (NULL) | cancelled | 2025-11-15 | Borrower found the book elsewhere and cancelled the request. |

## HIGHLIGHTS

- **Prozess:** Status ENUM steuert Workflow.
- **Historie:** Speichert Leihdauer und Zeitstempel.
- **Logik:** Rückgabe nicht vor Startdatum möglich.

# Entität: Rating

Ermöglicht gegenseitige Bewertungen zwischen Mitgliedern nach abgeschlossenen Ausleihen.

## SQL STATEMENT

```sql
CREATE TABLE rating (
    id BIGINT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    loan_id BIGINT NOT NULL,
    rater_id BIGINT NOT NULL,
    rated_member_id BIGINT NOT NULL,
    stars TINYINT NOT NULL,
    comment TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (loan_id) REFERENCES loan(id) ON DELETE CASCADE  ON UPDATE CASCADE,
    FOREIGN KEY (rater_id) REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (rated_member_id) REFERENCES member(id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT chk_rating_stars CHECK (stars BETWEEN 1 AND 5),
    CONSTRAINT uq_rating UNIQUE (loan_id, rater_id),

    INDEX idx_rating_rated (rated_member_id),
    INDEX idx_rating_stars (stars)

) ENGINE=InnoDB COMMENT='Mutual ratings after completed loan transactions';

INSERT INTO rating (loan_id, rater_id, rated_member_id, stars, comment, created_at) VALUES
(1, 1, 2, 5, 'Excellent borrower! Book was returned on time and in perfect condition.
Great communication throughout.', '2024-10-21 10:15:00')
```

bookexchange.rating: 10 Zeilen gesamt (exakt)

| # | 1 id | 2 loan_id | 3 rater_id | 4 rated_member_id | 5 stars | 6 comment | 7 created_at |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2 | 5 | Excellent borrower! Book was returned on time and in pe... | 2024-10-21 10:15:00 |
| 2 | 2 | 1 | 2 | 1 | 4 | Book was as described. Pickup was easy and flexible. Wo... | 2024-10-21 14:30:00 |
| 3 | 3 | 2 | 2 | 3 | 5 | Very reliable borrower. Returned the book a day early. H... | 2024-10-19 09:00:00 |
| 4 | 4 | 2 | 3 | 2 | 5 | Perfect transaction. Thank you! | 2024-10-19 11:20:00 |
| 5 | 5 | 3 | 3 | 4 | 3 | Book was returned a bit late, but borrower communicate... | 2024-11-14 16:30:00 |
| 6 | 6 | 3 | 4 | 3 | 4 | Good book condition. Pickup location was convenient. Wo... | 2024-11-14 18:00:00 |
| 7 | 7 | 4 | 4 | 6 | 5 | (NULL) | 2024-12-02 08:45:00 |
| 8 | 8 | 4 | 6 | 4 | 5 | Very friendly and accommodating lender. Fast response t... | 2024-12-02 10:00:00 |
| 9 | 9 | 5 | 6 | 7 | 4 | Good borrower. Book returned on time and in good condi... | 2024-11-24 15:20:00 |
| 10 | 10 | 5 | 7 | 6 | 5 | Amazing experience! The book was even better than exp... | 2024-11-24 16:45:00 |

## HIGHLIGHTS

- **Validierung:** Sterne auf 1-5 begrenzt.
- **Spam-Schutz:** Eine Bewertung pro Leihe.
- **Kopplung:** Rating direkt an Transaktionen gebunden.

# Teil II: Prozesslogik und Trigger

# Proaktive Integrität: Trigger zur Update-Validierung

## SQL STATEMENT

```
DELIMITER //
CREATE TRIGGER trg_availability_no_overlap_upd
BEFORE UPDATE ON availability
FOR EACH ROW
BEGIN
    IF NEW.status = 'active' THEN
        IF EXISTS (
            SELECT 1
            FROM availability a
            WHERE a.copy_id = NEW.copy_id
              AND a.status  = 'active'
              AND a.id     <> NEW.id
              AND (
                    NEW.available_from <= a.available_to
                  AND NEW.available_to   >= a.available_from
              )
        ) THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Overlapping active availability for this copy is not allowed (UPDATE).';
        END IF;
    END IF;
END;
//
DELIMITER ;
```

## ABLAUF

- Überwachung von Änderungen bei bestehenden Verfügbarkeiten (BEFORE UPDATE).
- Abgleich des neuen Zeitraums mit existierenden Daten auf Kollisionen.
- Zentraler Schutz vor nachträglichen Logikfehlern in der Stammdatenpflege.

# Kollisionsvermeidung: Automatisierte Prüfung bei Neuanlagen

## SQL STATEMENT

```sql
DELIMITER //
CREATE TRIGGER trg_availability_no_overlap_ins
BEFORE INSERT ON availability
FOR EACH ROW
BEGIN
    IF NEW.status = 'active' THEN
        IF EXISTS (
            SELECT 1
            FROM availability a
            WHERE a.copy_id = NEW.copy_id
              AND a.status  = 'active'
              AND (
                    NEW.available_from <= a.available_to
                  AND NEW.available_to   >= a.available_from
              )
        ) THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Overlapping active availability for this copy is not allowed (INSERT).';
        END IF;
    END IF;
END;
//
DELIMITER ;
```

## ABLAUF

- Automatische Prüfung bei jedem neuen Eintrag in der Tabelle availability.
- Verhindert proaktiv die Speicherung logisch widersprüchlicher Zeiträume.

# Transaktions-Sicherheit: Status-basierte Leihblockierung

## SQL STATEMENT

```sql
DELIMITER //
CREATE TRIGGER trg_loan_block_if_copy_not_available
BEFORE INSERT ON loan
FOR EACH ROW
BEGIN
    DECLARE v_copy_status         VARCHAR(20);
    DECLARE v_availability_status VARCHAR(20);

    SELECT status
    INTO v_availability_status
    FROM availability
    WHERE id = NEW.availability_id;

    IF v_availability_status <> 'active' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loan not allowed: availability is not active.';
    END IF;

    SELECT bc.status
    INTO v_copy_status
    FROM availability a
    JOIN book_copy bc ON bc.id = a.copy_id
    WHERE a.id = NEW.availability_id;

    IF v_copy_status <> 'available' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loan not allowed: copy is not available (status != available).';
    END IF;

END;
//
DELIMITER ;
```

## ABLAUF

- Verknüpfung der Leihanfrage mit dem physischen Status des Exemplars.
- Blockiert Transaktionen, wenn der Status nicht available ist.
- Erzwingt den korrekten Prozess-Workflow (Status-Enforcement).

# Datenintegrität: Verhinderung von Selbst-Ausleihen (INSERT)

## SQL STATEMENT

```
DELIMITER //
CREATE TRIGGER trg_loan_no_self_loan
BEFORE INSERT ON loan
FOR EACH ROW
BEGIN
    IF NEW.borrower_id = NEW.lender_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Loan not allowed: lender and borrower must be different members.';
    END IF;
END;
//
DELIMITER ;
```

## ABLAUF

- Überwachung neuer Leihanfragen in der Tabelle loan (BEFORE INSERT)
- Vergleich von borrower_id und lender_id, um identische Mitglieder zu erkennen.
- Blockiert den Vorgang bei Übereinstimmung, da Mitglieder keine Bücher an sich selbst verleihen dürfen.

# Datenintegrität: Verhinderung von Selbst-Ausleihen (UPDATE)

## SQL STATEMENT

```
DELIMITER //
CREATE TRIGGER trg_loan_no_self_loan_upd
BEFORE UPDATE ON loan
FOR EACH ROW
BEGIN
    IF NEW.borrower_id = NEW.lender_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Loan not allowed: lender and borrower must be different members.';
    END IF;
END;
//
DELIMITER ;
```

## ABLAUF

- Überwachung von Änderungen an bestehenden Leihtransaktionen (BEFORE UPDATE).
- Prüfung der neuen Rollenwerte (NEW.borrower_id und NEW.lender_id).
- Unterbindet unzulässige Änderungen und erzwingt die Trennung von Verleiher und Ausleiher.

# Teil III: Funktionale Validierung

# Test Szenario: Happy Path

- **Ziel:** Prüfung der relationalen Integrität von der Suche bis zur Bewertung.
- **Akteure:** Anna (Verleiherin) und Ben (Ausleiher).

1. Verfügbare Bücher anzeigen
2. Leihanfrage stellen
3. Anfrage annehmen & Ausleihe starten
4. Rückgabe & Abschluss
5. Gegenseitige Bewertung

# Schritt 1 – Verfügbare Bücher anzeigen

## SQL STATEMENT

```sql
SELECT
    b.id AS book_id,
    b.title,
    a.last_name AS author_last_name,
    g.name AS genre,
    l.city,
    av.id AS availability_id,
    av.max_duration_days
FROM book b
JOIN book_author ba ON b.id = ba.book_id
JOIN author a ON ba.author_id = a.id
JOIN book_genre bg ON b.id = bg.book_id
JOIN genre g ON bg.genre_id = g.id
JOIN book_copy bc ON b.id = bc.book_id
JOIN availability av ON bc.id = av.copy_id
JOIN location l ON av.location_id = l.id
WHERE l.city = 'Berlin' AND bc.status = 'available' AND av.status = 'active';
```

book (3r × 7c)

| # | 1 book_id | 2 title | 3 author_last_name | 4 genre | 5 city | 6 availability_id | 7 max_duration_days |
|---|---|---|---|---|---|---|---|
| 1 | 1 | Der Prozess | Kafka | Roman | Berlin | 1 | 21 |
| 2 | 4 | La Sombra del Viento | Ruiz Zafón | Historischer Roman | Berlin | 4 | 30 |
| 3 | 8 | Nineteen Eighty-Four | Orwell | Dystopie | Berlin | 8 | 7 |

## ABLAUF

- Mitglied Ben sucht nach verfügbaren Büchern in Berlin.

# Schritt 2 – Leihanfrage stellen

## SQL STATEMENT

```
INSERT INTO loan (availability_id, lender_id, borrower_id, status)
VALUES (1, 1, 2, 'requested');

SET @loan_id := LAST_INSERT_ID();
```

## ABLAUF

- Mitglied Ben sendet eine Anfrage für das ausgewählte Buch.
- In der Datenbank entstehe ein neuer Loan-Eintrag mit dem Status ‘requested‘.

loan (1r × 11c)

| # | 1 id | 2 availability_id | 3 lender_id | 4 borrower_id | 5 request_date | 6 start_date | 7 planned_end_date | 8 actual_end_date | 9 status | 10 cancellation_d... | 11 cancellation_rea... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 11 | 1 | 1 | 2 | 2026-02-11 20:33:30 | (NULL) | (NULL) | (NULL) | requested | (NULL) | (NULL) |

# Schritt 3 – Anfrage annehmen & Ausleihe starten

## SQL STATEMENT

```sql
UPDATE loan
SET status = 'active', start_date = CURRENT_DATE,
    planned_end_date = DATE_ADD(CURRENT_DATE, INTERVAL 14 DAY)
WHERE id = @loan_id;

UPDATE book_copy
SET status = 'on_loan'
WHERE id = 1;
```

## ABLAUF

- Mitglied Anna prüft die Anfrage und akzeptiert sie.
- Loan-Status wechselt von ‘requested‘ zu ‘active‘.
- Startdatum und geplantes Enddatum werden gesetzt
- Das physische Buchexemplar wird auf ‘on_loan‘ gesetzt, wodurch weitere Ausleihanfragen für dieselbe Kopie verhindert werden.

loan (1r × 11c)

| # | 1 id | 2 availability_id | 3 lender_id | 4 borrower_id | 5 request_date | 6 start_date | 7 planned_end_date | 8 actual_end_date | 9 status | 10 cancellation_d... | 11 cancellation_rea... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 11 | 1 | 1 | 2 | 2026-02-11 20:33:30 | 2026-02-11 | 2026-02-25 | (NULL) | active | (NULL) | (NULL) |

book_copy (1r × 8c)

| # | 1 id | 2 book_id | 3 owner_id | 4 condition_code | 5 condition_description | 6 acquisition_date | 7 notes | 8 status |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | good | Some dog-eared pages on chapters 3 and 7. Minor wear ... | 2020-05-15 | Purchased at a local bookstore. Has my name written insi... | on_loan |

# Schritt 4 – Rückgabe & Abschluss

## SQL STATEMENT

```sql
UPDATE loan
SET status = 'returned', actual_end_date = CURRENT_DATE
WHERE id = @loan_id;

UPDATE book_copy
SET status = 'available'
WHERE id = 1;
```

## ABLAUF

- Mitglied Ben gibt das Buch fristgerecht zurück.
- Loan-Status wird auf ‘returned‘ gesetzt.
- Rückgabedatum wird eingetragen.
- Buchkopie wird wieder auf ‘available‘ gesetzt und kann wieder von anderen Mitgliedern ausgeliehen werden.

loan (1r × 11c)

| # | 1 id | 2 availability_id | 3 lender_id | 4 borrower_id | 5 request_date | 6 start_date | 7 planned_end_date | 8 actual_end_date | 9 status | 10 cancellation_d... | 11 cancellation_rea... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 11 | 1 | 1 | 2 | 2026-02-11 20:33:30 | 2026-02-11 | 2026-02-25 | 2026-02-11 | returned | (NULL) | (NULL) |

book_copy (1r × 8c)

| # | 1 id | 2 book_id | 3 owner_id | 4 condition_code | 5 condition_description | 6 acquisition_date | 7 notes | 8 status |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | good | Some dog-eared pages on chapters 3 and 7. Minor wear ... | 2020-05-15 | Purchased at a local bookstore. Has my name written insi... | available |

# Schritt 5 – Gegenseitige Bewertung

## SQL STATEMENT

```
INSERT INTO rating (loan_id, rater_id, rated_member_id, stars, comment)
VALUES (@loan_id, 1, 2, 5, 'Very reliable borrower, book came back in top condition!');

INSERT INTO rating (loan_id, rater_id, rated_member_id, stars, comment)
VALUES (@loan_id, 2, 1, 5, 'Super straightforward handover, thank you!');
```

## ABLAUF

- Nach erfolgreicher Rückgabe geben beide Mitglieder eine Bewertung ab.
- Verleiher (Anna) bewertet den Ausleiher (Ben).
- Ausleiher (Ben) bewertet den Verleiher Anna).
- Beide Bewertungen beziehen sich zwingend auf denselben Loan.

rating (2r × 7c)

| # | 1 id | 2 loan_id | 3 rater_id | 4 rated_member_id | 5 stars | 6 comment | 7 created_at |
|---|---|---|---|---|---|---|---|
| 1 | 13 | 12 | 1 | 2 | 5 | Very reliable borrower, book came back in top condition! | 2026-02-11 20:40:28 |
| 2 | 14 | 12 | 2 | 1 | 5 | Super straightforward handover, thank you! | 2026-02-11 20:40:28 |

# Test Szenario: Validierung der Datenintegrität

- **Ziel:** Überprüfung der Wirksamkeit von CHECK- und UNIQUE-Constraints sowie der referenziellen Integrität im Bewertungssystem.
- **Akteure:** Anna (Verleiherin) und Ben (Ausleiher).

1. Bestehende Bewertungen prüfen
2. Ungültige Bewertungen abgeben
3. Doppelte Bewertung einreichen
4. Integritätsprüfung nach Fehlerfall

# Schritt 1 – Bestehende Bewertungen prüfen

## SQL STATEMENT

```sql
SELECT
    r.id,
    r.loan_id,
    r.rater_id,
    r.rated_member_id,
    r.stars,
    r.comment,
    r.created_at
FROM rating r
WHERE r.loan_id = 1;
```

## ABLAUF

- Vor dem Test wird überprüft, welche Bewertungen bereits für den ausgewählten Loan existieren.
- SELECT-Statement filtert alle Datensätze der Entität rating anhand der loan_id.
- Ziel ist es, den aktuellen Zustand der Datenbank vor dem fehlerhaften INSERT sichtbar zu machen und diesen als Referenzwert zu nutzen.

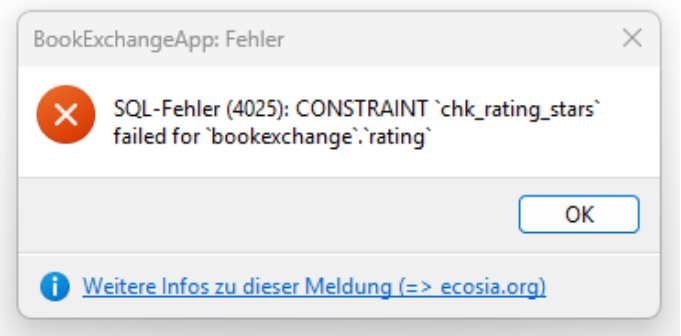
rating (2r × 7c)

| # | 1 id | 2 loan_id | 3 rater_id | 4 rated_member_id | 5 stars | 6 comment | 7 created_at |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2 | 5 | Excellent borrower! Book was returned on time and in pe... | 2024-10-21 10:15:00 |
| 2 | 2 | 1 | 2 | 1 | 4 | Book was as described. Pickup was easy and flexible. Wo... | 2024-10-21 14:30:00 |

# Schritt 2 – Ungültige Bewertung abgeben

## SQL STATEMENT

```
INSERT INTO rating (loan_id, rater_id, rated_member_id, stars, comment)
VALUES (1, 1, 2, 6, 'Test: invalid stars (should fail)');
```




## ABLAUF

- Ein Mitglied versucht eine Bewertung mit ungültiger Sternzahl abzugeben (z. B. 6 Sterne)
- System prüft die Eingabe gegen den definierten CHECK-Constraint (stars BETWEEN 1 AND 5)
- Datenbank lehnt den INSERT-Vorgang ab. Somit wird kein neuer Datensatz in der Entität rating gespeichert.

# Schritt 3 - Doppelte Bewertung einreichen

## SQL STATEMENT

```sql
INSERT INTO rating (loan_id, rater_id, rated_member_id, stars, comment)
VALUES (1, 1, 2, 5, 'Test: duplicate rating (should fail)');
```

## ABLAUF

- Ein Mitglied versucht denselben Loan ein zweites Mal zu bewerten.
- UNIQUE-Constraint (loan_id, rater_id) wird verletzt.
- Datenbank blockiert den Eintrag automatisch. Somit ist weiterhin nur eine Bewertung pro Nutzer möglich.

# Schritt 4 – Integritätsprüfung nach Fehlerfall

## SQL STATEMENT

```sql
SELECT
    COUNT(*) AS rating_count_after
FROM rating
WHERE loan_id = 1;
```

rating (1r × 1c)

| # | 1 rating_count_after |
|---|---|
| 1 | 2 |

## ABLAUF

- Nach dem fehlerhaften INSERT wird überprüft, ob sich die Anzahl der Bewertungen verändert hat
- Abfrage bestätigt: Keine zusätzlichen Datensätze wurden erzeugt. Die Constraints funktionieren wie vorgesehen.

# Edge Cases

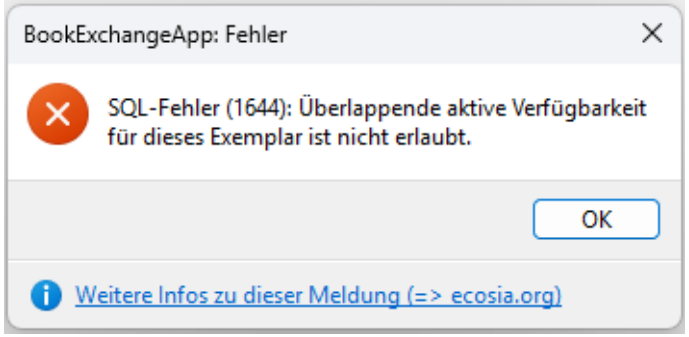
- **Zentrales Ziel:** Nachweis der System-Resilienz und Datenintegrität durch Verifizierung der Sicherheitsmechanismen (Trigger/Constraints) bei logischen Fehlbedingungen und Grenzwerten.

1. Schutz vor zeitlichen Überschneidungen
2. Validierung von Zeitintervallen
3. Erzwingung globaler Format-Standards
4. Schutz vor Race Conditions bei Ausleihen
5. Schutz vor verwaisten Daten
6. Integrität räumlicher Koordinatensysteme

# Edge Case 1 – Schutz vor zeitlichen Überschneidungen

## SQL STATEMENT

```sql
INSERT INTO availability (copy_id, location_id, available_from, available_to,
                          max_duration_days, shipping_possible, status)
VALUES (1, 1, '2024-06-01', '2024-08-01', 14, FALSE, 'active');
```




## ABLAUF

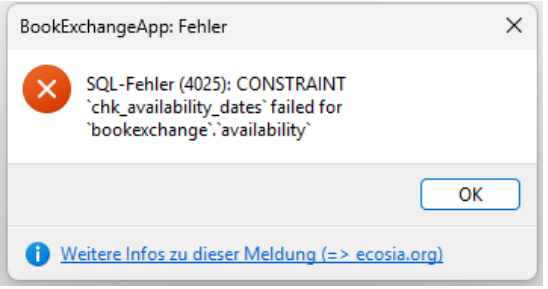
- Versuch, ein zweites Verfügbarkeitsfenster für dasselbe Buch-Exemplar anzulegen.
- Trigger trg_availability_no_overlap_ins prüft auf zeitliche Kollisionen.
- Datenbank blockiert den INSERT, da ein physisches Objekt zeitgleich nur an einem Ort sein kann.

# Edge Case 2 – Validierung von Zeitintervallen

## SQL STATEMENT

```sql
INSERT INTO availability (copy_id, location_id, available_from, available_to, max_duration_days)
VALUES (2, 2, '2026-01-01', '2026-02-01', 0);
```

```sql
INSERT INTO availability (copy_id, location_id, available_from, available_to)
VALUES (2, 2, '2026-02-01', '2026-01-01');
```



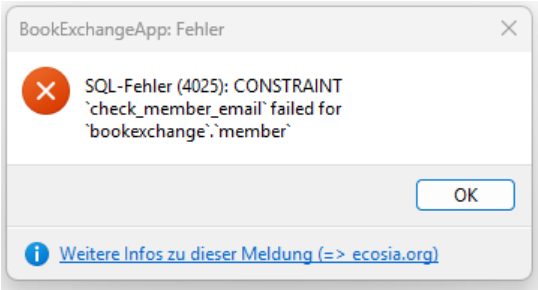
## ABLAUF

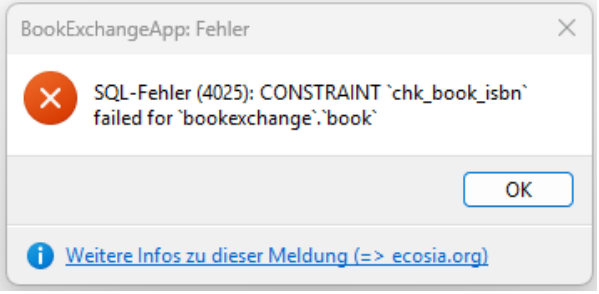
- Erzwingung logischer Zeitabfolgen: Das Enddatum muss nach dem Startdatum liegen.
- Sicherstellung einer positiven Leihdauer (max_duration_days).
- CHECK-Constraints verhindern fehlerhafte Berechnungen in der Applikationsebene.

# Edge Case 3 – Erzwingung globaler Format-Standards

## SQL STATEMENT

```sql
INSERT INTO member (first_name, last_name, email_address, password_hash)
VALUES ('Test', 'User', 'falsche-email@com', 'hash');

INSERT INTO book (title, isbn)
VALUES ('Invalid Book', '12345');
```



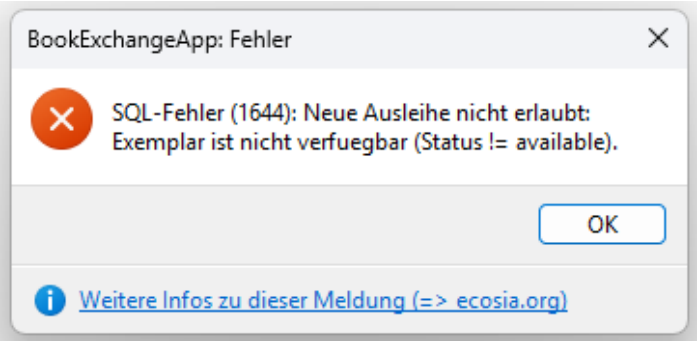

## ABLAUF

- Validierung kritischer Datenformate direkt beim Datenbank-Eintrag.
- Prüfung des E-Mail-Basisformats und der ISBN-10/ISBN-13 Norm.
- Erhöht die Datenqualität unabhängig von der Eingabemaske im Frontend.

# Edge Case 4 – Schutz vor Race Conditions bei Ausleihen

## SQL STATEMENT

```sql
UPDATE book_copy SET status = 'on_loan' WHERE id = 1;

INSERT INTO loan (availability_id, lender_id, borrower_id, status)
VALUES (1, 1, 3, 'requested');
```



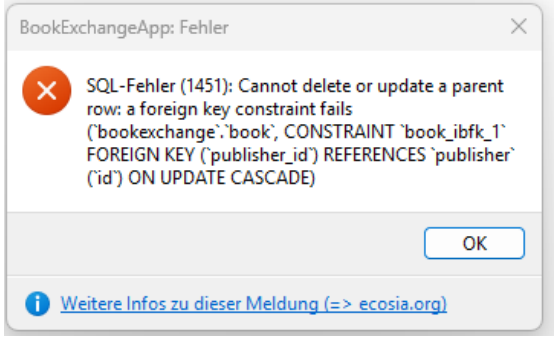

## ABLAUF

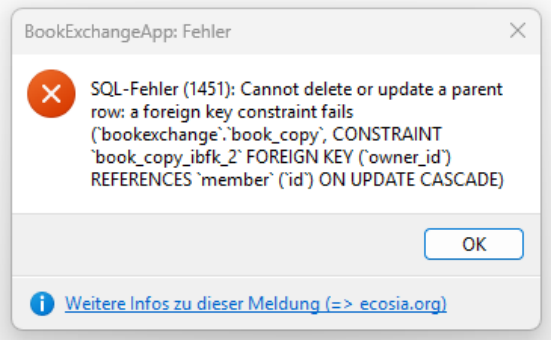
- Simulation einer Doppelbuchung: Anfrage für ein bereits verliehenes Buch (on_loan).
- Trigger trg_loan_block_if_copy_not_available prüft den Echtzeit-Status des Exemplars.
- Datenbank unterbindet die Erstellung des Leihvorgangs zum Schutz vor Mehrfachreservierung.

# Edge Case 5 – Schutz vor verwaisten Daten

## SQL STATEMENT

```sql
DELETE FROM publisher
WHERE id = 1;

DELETE FROM member
WHERE id = 1;
```



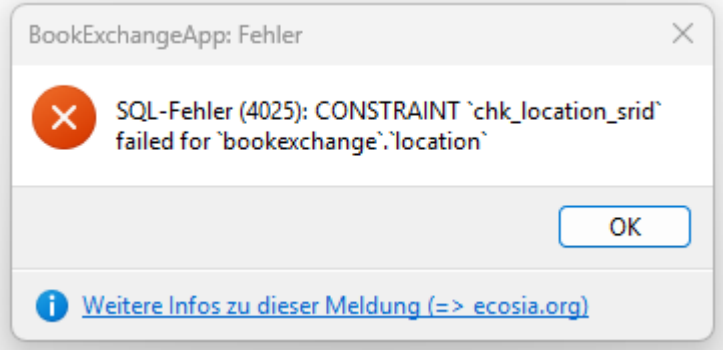

## ABLAUF

- **Test 1:** Versuch, einen Verlag zu löschen, dem noch Bücher zugeordnet sind.
- **Test 2:** Versuch, ein Mitglied zu löschen, das noch aktive Ausleihen in der Historie hat.
- Die Datenbank blockiert beide Vorgänge durch ON DELETE RESTRICT.
- Verwaiste Datensätze werden verhindert. Somit bleibt die Datenhistorie konsistent.

# Edge Case 6 – Integrität räumlicher Koordinatensysteme

## SQL STATEMENT

```
INSERT INTO location (member_id, street, house_number, postal_code,
                      city, country_code, coordinates, label)
VALUES (1, 'Teststraße', '1', '99999', 'Teststadt', 'DE',
        ST_PointFromText('POINT(13.0000 52.0000)', 3857),
        'home');
```




## ABLAUF

- Versuch, Geodaten mit falschem Koordinatensystem zu speichern.
- Erzwingung des Standards SRID 4326 (WGS 84).
- Gewährleistung präziser, GPS-konformer Standortdaten.

# Teil IV: Data Mart und Analysen

# Analytische Sicht auf Buch-KPIs

dm_book_performance (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW dm_book_performance AS
SELECT
    b.id AS book_id,
    b.title,
    MIN(g.name) AS primary_genre,
    COUNT(DISTINCT l.id) AS total_loans,
    ROUND(AVG(DATEDIFF(l.actual_end_date, l.start_date)), 1) AS avg_loan_duration_days,
    ROUND(AVG(r.stars), 2) AS avg_rating
FROM book b
LEFT JOIN book_genre bg ON b.id = bg.book_id
LEFT JOIN genre g ON bg.genre_id = g.id
LEFT JOIN book_copy bc ON b.id = bc.book_id
LEFT JOIN availability av ON bc.id = av.copy_id
LEFT JOIN loan l ON av.id = l.availability_id
LEFT JOIN rating r ON l.id = r.loan_id
GROUP BY b.id, b.title;
```

## ABLAUF

- Aggregation von Leihfrequenzen und Nutzerbewertungen auf Werk-Ebene. Berechnung qualitativer (Rating) und quantitativer (Total Loan) KPIs.
- Bedeutung: Erstellt ein multidimensionales Bild der Buchbeliebtheit ohne Beeinträchtigung der operativen Performance.

# Bestseller-Ranking und Qualitätskontrolle

dm_book_performance (2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    book_id,
    title,
    primary_genre,
    total_loans,
    avg_loan_duration_days,
    avg_rating
FROM dm_book_performance
ORDER BY book_id;
```

```sql
SELECT
    title,
    primary_genre,
    total_loans,
    avg_rating
FROM dm_book_performance
ORDER BY total_loans DESC
LIMIT 5;
```

## ABLAUF

- Analyse der durchschnittlichen Bewertung pro Genre und Ranking der „Top 5“ Werke nach Ausleihanzahl.
- **Relevanz:** Liefert Business-Insights zur Bestandsoptimierung und zur datengestützten Steuerung von Empfehlungsalgorithmen.

book (11r × 6c) | book (5r × 4c)

| # | 1 book_id | 2 title | 3 primary_genre | 4 total_loans | 5 avg_loan_duration_days | 6 avg_rating |
|---|---|---|---|---|---|---|
| 1 | 1 | Der Prozess | Roman | 7 | 5,4 | 4,83 |
| 2 | 2 | 1Q84 | Roman | 1 | 13,0 | 5,0 |
| 3 | 3 | The Handmaids Tale | Dystopie | 1 | 12,0 | 3,5 |
| 4 | 4 | La Sombra del Viento | Historischer Roman | 1 | 26,0 | 5,0 |
| 5 | 5 | Lamica geniale | Roman | 1 | 13,0 | 4,5 |
| 6 | 6 | Sérotonine | Gegenwartsliteratur | 1 | (NULL) | (NULL) |
| 7 | 7 | Harry Potter and the Philosophers Stone | Fantasy | 1 | (NULL) | (NULL) |
| 8 | 8 | Nineteen Eighty-Four | Dystopie | 1 | (NULL) | (NULL) |
| 9 | 9 | Eine kurze Geschichte der Menschheit | Sachbuch | 1 | (NULL) | (NULL) |
| 10 | 10 | Unterleuten | Gegenwartsliteratur | 1 | (NULL) | (NULL) |
| 11 | 11 | The Great Gatsby | (NULL) | 0 | (NULL) | (NULL) |

book (11r × 6c) | book (5r × 4c)

| # | 1 title | 2 primary_genre | 3 total_loans | 4 avg_rating |
|---|---|---|---|---|
| 1 | Der Prozess | Roman | 7 | 4,83 |
| 2 | The Handmaids Tale | Dystopie | 1 | 3,5 |
| 3 | Sérotonine | Gegenwartsliteratur | 1 | (NULL) |
| 4 | Eine kurze Geschichte der Menschheit | Sachbuch | 1 | (NULL) |
| 5 | La Sombra del Viento | Historischer Roman | 1 | 5,0 |

# Nutzer-Statistiken (Struktur und Trust Score)

dm_member_stats (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW dm_member_stats AS
SELECT
    m.id AS member_id,
    MIN(loc.city) AS primary_city,
    COUNT(DISTINCT bc.id) AS books_offered,
    COUNT(DISTINCT l_borrowed.id) AS total_borrowed,
    COUNT(DISTINCT r.id) AS ratings_received,
    ROUND(AVG(r.stars), 2) AS trust_score
FROM member m
LEFT JOIN book_copy bc ON m.id = bc.owner_id
LEFT JOIN loan l_borrowed ON m.id = l_borrowed.borrower_id
LEFT JOIN rating r ON m.id = r.rated_member_id
LEFT JOIN location loc ON m.id = loc.member_id
GROUP BY m.id;
```

## ABLAUF

- Kombination von Gebotsaktivität, Leihhistorie und erhaltenem Feedback in einem Reputationsprofil. Berechnung des gewichteten Trust Scores.
- Bedeutung: Konsolidiert Verhaltensdaten zu einem greifbaren Metrik-System für das Community-Management.

# Reputations-Monitoring und Aktivitätsprofile

dm_member_stats(2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    member_id,
    primary_city,
    books_offered,
    total_borrowed,
    ratings_received,
    trust_score
FROM dm_member_stats
ORDER BY member_id;
```

```sql
SELECT
    member_id,
    primary_city,
    books_offered,
    total_borrowed,
    ratings_received,
    trust_score
FROM dm_member_stats
WHERE ratings_received >= 3
  AND trust_score >= 4.5
  AND (books_offered > 0 OR total_borrowed > 0)
ORDER BY trust_score DESC, ratings_received DESC;
```

## ABLAUF

- Identifikation von „High Trust“-Mitgliedern basierend auf Aktivitätsschwellenwerten und Bewertungsschnitt.
- Relevanz: Dient zur Qualitätssicherung innerhalb der Community und ermöglicht risikobasiertes Monitoring der Nutzerreputation.

member (10r × 6c) | member (2r × 6c)

| # | 1 member_id | 2 primary_city | 3 books_offered | 4 total_borrowed | 5 ratings_received | 6 trust_score |
|---|---|---|---|---|---|---|
| 1 | 1 | Berlin | 2 | 2 | 3 | 4,67 |
| 2 | 2 | Hamburg | 2 | 4 | 4 | 5,0 |
| 3 | 3 | München | 1 | 4 | 2 | 4,5 |
| 4 | 4 | Berlin | 1 | 2 | 2 | 4,0 |
| 5 | 5 | Köln | 0 | 0 | 0 | (NULL) |
| 6 | 6 | Frankfurt am Main | 1 | 1 | 2 | 5,0 |
| 7 | 7 | Paris | 1 | 1 | 1 | 4,0 |
| 8 | 8 | Berlin | 0 | 0 | 0 | (NULL) |
| 9 | 9 | Hamburg | 1 | 1 | 0 | (NULL) |
| 10 | 10 | Berlin | 1 | 1 | 0 | (NULL) |

member (10r × 6c) | member (2r × 6c)

| # | 1 member_id | 2 primary_city | 3 books_offered | 4 total_borrowed | 5 ratings_received | 6 trust_score |
|---|---|---|---|---|---|---|
| 1 | 2 | Hamburg | 2 | 4 | 4 | 5,0 |
| 2 | 1 | Berlin | 2 | 2 | 3 | 4,67 |

# Regionales Reporting (Struktur und Gruppierung)

vw_report_loans_per_city_month (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW vw_report_loans_per_city_month AS
SELECT
    DATE_FORMAT(l.start_date, '%Y-%m-01') AS month_start,
    loc.city,
    g.name AS genre_name,
    COUNT(*) AS loans_count
FROM loan l
JOIN availability av ON l.availability_id = av.id
JOIN book_copy bc ON av.copy_id = bc.id
JOIN book b ON bc.book_id = b.id
JOIN book_genre bg ON b.id = bg.book_id
JOIN genre g ON bg.genre_id = g.id
JOIN location loc ON av.location_id = loc.id
WHERE l.start_date IS NOT NULL
GROUP BY
    DATE_FORMAT(l.start_date, '%Y-%m-01'),
    loc.city,
    g.name;
```

## ABLAUF

- Transformation der Einzeltransaktionen in ein monatliches Zeitreihen-Format, gruppiert nach geografischen Standorten und Genres.
- Bedeutung: Spezialisierte Reporting-View für Dashboard-Visualisierungen und Trendanalysen.

# Zeitliche und Geografische Hotspot-Analyse

vw_report_loans_per_city_month (2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    month_start,
    city,
    genre_name,
    loans_count
FROM vw_report_loans_per_city_month
ORDER BY month_start, city, genre_name;
```

```sql
SELECT
    month_start,
    city,
    SUM(loans_count) AS total_loans_in_city
FROM vw_report_loans_per_city_month
GROUP BY month_start, city
ORDER BY month_start, total_loans_in_city DESC;
```

## ABLAUF

- Analyse der monatlichen Ausleihvorgängen pro Stadt zur Identifikation von saisonalen und regionalen Mustern.
- Relevanz: Ermöglicht die Identifikation von regionalen Hotspots und liefert die Entscheidungsgrundlage für lokale Marketing-Maßnahmen oder Community-Events.

location (9r × 4c) | location (9r × 3c)

| # | 1 month_start | 2 city | 3 genre_name | 4 loans_count |
|---|---|---|---|---|
| 1 | 2024-10-01 | Berlin | Roman | 1 |
| 2 | 2024-10-01 | Hamburg | Roman | 1 |
| 3 | 2024-11-01 | Berlin | Historischer Roman | 1 |
| 4 | 2024-11-01 | Frankfurt am Main | Roman | 1 |
| 5 | 2024-11-01 | München | Dystopie | 1 |
| 6 | 2025-10-01 | Berlin | Dystopie | 1 |
| 7 | 2025-11-01 | Berlin | Sachbuch | 1 |
| 8 | 2025-11-01 | Paris | Gegenwartsliteratur | 1 |
| 9 | 2026-02-01 | Berlin | Roman | 3 |

location (9r × 4c) | location (9r × 3c)

| # | 1 month_start | 2 city | 3 total_loans_in_city |
|---|---|---|---|
| 1 | 2024-10-01 | Berlin | 1 |
| 2 | 2024-10-01 | Hamburg | 1 |
| 3 | 2024-11-01 | Berlin | 1 |
| 4 | 2024-11-01 | Frankfurt am Main | 1 |
| 5 | 2024-11-01 | München | 1 |
| 6 | 2025-10-01 | Berlin | 1 |
| 7 | 2025-11-01 | Berlin | 1 |
| 8 | 2025-11-01 | Paris | 1 |
| 9 | 2026-02-01 | Berlin | 3 |

# Mitglieder-Dimension (Struktur und Dimension)

vw_dm_member_dim (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW vw_dm_member_dim AS
SELECT
    m.id AS member_id,
    m.first_name,
    m.last_name,
    m.email_address,
    m.registration_date,
    YEAR(m.registration_date) AS registration_year,
    m.status,
    m.role
FROM member m;
```

## ABLAUF

- Aufbereitung der Mitglieder-Stammdaten inklusive Rollen und Status. Extraktion des Registrierungsjahres zur Ermöglichung von Kohortenanalysen.
- Bedeutung: Reduziert die Redundanz bei Nutzerabfragen und stellt bereinigte Profile für administrative Auswertungen bereit.

# Mitgliedsstruktur und Wachstums-Metriken

vw_dm_member_dim (2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    member_id,
    first_name,
    last_name,
    registration_year,
    status,
    role
FROM vw_dm_member_dim
ORDER BY member_id;
```

```sql
SELECT
    registration_year,
    COUNT(*) AS members_registered
FROM vw_dm_member_dim
GROUP BY registration_year
ORDER BY registration_year;
```

vw_dm_member_dim (10r × 6c) | vw_dm_member_dim (1r × 2c)

| # | 1 member_id | 2 first_name | 3 last_name | 4 registration_year | 5 status | 6 role |
|---|---|---|---|---|---|---|
| 1 | 1 | Anna | Müller | 2.026 | active | member |
| 2 | 2 | Ben | Schmidt | 2.026 | active | member |
| 3 | 3 | Carla | Schneider | 2.026 | active | admin |
| 4 | 4 | David | Fischer | 2.026 | active | member |
| 5 | 5 | Eva | Weber | 2.026 | inactive | member |
| 6 | 6 | Felix | Meyer | 2.026 | active | member |
| 7 | 7 | Greta | Wagner | 2.026 | active | member |
| 8 | 8 | Hans | Becker | 2.026 | suspended | member |
| 9 | 9 | Ida | Hoffmann | 2.026 | active | member |
| 10 | 10 | Jan | Schulz | 2.026 | active | member |

## ABLAUF

- Überprüfung der Nutzerprofile und statistische Auswertung des Mitgliederwachstums pro Kalenderjahr.
- Relevanz: Ermöglicht die Segmentierung der Nutzerbasis zur Identifikation von Wachstumsraten und bildet die Basis für Maßnahmen zur langfristigen Mitgliederbindung und zur Analyse der Nutzerflukation.

vw_dm_member_dim (10r × 6c) | vw_dm_member

| # | 1 registration_year | 2 members_registered |
|---|---|---|
| 1 | 2.026 | 10 |

# Buch-Dimension (Struktur und Definition)

vw_dm_book_dim (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW vw_dm_book_dim AS
SELECT
    b.id AS book_id,
    b.title,
    b.subtitle,
    b.isbn,
    b.publication_year,
    p.name AS publisher_name,
    lang.name AS language_name,
    MIN(g.name)        AS primary_genre
FROM book b
LEFT JOIN publisher p ON b.publisher_id = p.id
LEFT JOIN language lang ON b.language_id = lang.id
LEFT JOIN book_genre bg ON b.id = bg.book_id
LEFT JOIN genre g ON bg.genre_id = g.id
GROUP BY b.id, b.title, b.subtitle, b.isbn, b.publication_year, p.name, lang.name;
```

## ABLAUF

- Konsolidierung von Buchtiteln mit ihren zugehörigen Verlagen, Sprachen und Genres. Aggregation von Multi-Genre-Zuweisungen auf ein primäres Genre (MIN-Logik).
- Bedeutung: Dient als zentrale Stammdatenquelle für Katalogreports, um komplexe Joins in der Analysephase zu vermeiden.

# Katalog-Vervollständigung und Bestandsverteilung

vw_dm_book_dim (2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    book_id,
    title,
    publisher_name,
    language_name,
    primary_genre
FROM vw_dm_book_dim
ORDER BY book_id;
```

```sql
SELECT
    language_name,
    publisher_name,
    COUNT(*) AS book_count
FROM vw_dm_book_dim
GROUP BY language_name, publisher_name
ORDER BY language_name, book_count DESC;
```

## ABLAUF

- Validierung der Katalog-Vollständigkeit und Analyse der Bestandsverteilung nach Sprache und Verlag.
- Relevanz: Verifiziert die korrekte Auflösung der Fremdschlüssel-Relationen und gewährleistet die Konsistenz der Stammdaten innerhalb des analytischen Layers.

book (11r × 5c) | language (7r × 3c)

| # | 1 book_id | 2 title | 3 publisher_name | 4 language_name | 5 primary_genre |
|---|---|---|---|---|---|
| 1 | 1 | Der Prozess | Suhrkamp Verlag | Deutsch | Roman |
| 2 | 2 | 1Q84 | Kiepenheuer & Witsch | Deutsch | Roman |
| 3 | 3 | The Handmaids Tale | Penguin Random House | English | Dystopie |
| 4 | 4 | La Sombra del Viento | Fischer Verlag | Español | Historischer Roman |
| 5 | 5 | Lamica geniale | Suhrkamp Verlag | Italiano | Roman |
| 6 | 6 | Sérotonine | Gallimard | Français | Gegenwartsliteratur |
| 7 | 7 | Harry Potter and the Philosophers Stone | Penguin Random House | English | Fantasy |
| 8 | 8 | Nineteen Eighty-Four | Penguin Random House | English | Dystopie |
| 9 | 9 | Eine kurze Geschichte der Menschheit | C.H. Beck | Deutsch | Sachbuch |
| 10 | 10 | Unterleuten | Suhrkamp Verlag | Deutsch | Gegenwartsliteratur |
| 11 | 11 | The Great Gatsby | Penguin Random House | English | (NULL) |

book (11r × 5c) | language (7r × 3c)

| # | 1 language_name | 2 publisher_name | 3 book_count |
|---|---|---|---|
| 1 | Deutsch | Suhrkamp Verlag | 2 |
| 2 | Deutsch | C.H. Beck | 1 |
| 3 | Deutsch | Kiepenheuer & Witsch | 1 |
| 4 | English | Penguin Random House | 4 |
| 5 | Español | Fischer Verlag | 1 |
| 6 | Français | Gallimard | 1 |
| 7 | Italiano | Suhrkamp Verlag | 1 |

# Leih-Fakten (Struktur und Kennzahlen)

vw_dm_loan_fact (1/2 – Definition)

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW vw_dm_loan_fact AS
SELECT
    l.id AS loan_id,
    l.availability_id,
    a.copy_id,
    bc.book_id,
    l.lender_id,
    l.borrower_id,
    l.request_date,
    l.start_date,
    l.planned_end_date,
    l.actual_end_date,
    l.status,
    CASE
        WHEN l.start_date IS NOT NULL AND l.planned_end_date IS NOT NULL
        THEN DATEDIFF(l.planned_end_date, l.start_date)
        ELSE NULL
    END AS planned_duration_days,
    CASE
        WHEN l.start_date IS NOT NULL AND l.actual_end_date IS NOT NULL
        THEN DATEDIFF(l.actual_end_date, l.start_date)
        ELSE NULL
    END AS actual_duration_days,
    CASE
        WHEN l.status = 'overdue'
          OR (l.actual_end_date  IS NOT NULL
              AND l.planned_end_date IS NOT NULL
              AND l.actual_end_date > l.planned_end_date)
          OR (l.actual_end_date  IS NULL
              AND l.planned_end_date IS NOT NULL
              AND l.planned_end_date < CURRENT_DATE())
        THEN 1 ELSE 0
    END AS is_overdue
FROM loan         l
JOIN availability a ON l.availability_id = a.id
JOIN book_copy bc ON a.copy_id = bc.id;
```

## ABLAUF

- Zusammenführung aller Transaktionsdaten in einer zentralen Faktentabelle. Integration berechneter Metriken wie Soll-Ist-Dauer und automatisierte Overdue-Flags.
- Bedeutung: Zentrales Element für prozessorientierte Abfragen. Optimiert auf schnelle Berechnungen von Zeitintervallen.

# Prozess-Monitoring und Leihdauer-Validierung

vw_dm_loan_fact (2/2 – Analyse & Ergebnisse)

## SQL STATEMENT

```sql
SELECT
    loan_id,
    lender_id,
    borrower_id,
    start_date,
    planned_end_date,
    actual_end_date,
    planned_duration_days,
    actual_duration_days,
    is_overdue
FROM vw_dm_loan_fact
ORDER BY loan_id;
```

```sql
SELECT
    ROUND(AVG(actual_duration_days), 2) AS avg_actual_duration_days
FROM vw_dm_loan_fact
WHERE actual_duration_days IS NOT NULL;
```

vw_dm_loan_fact (16r × 9c) | vw_dm_loan_fact (1r × 1c)

| # | 1 loan_id | 2 lender_id | 3 borrower_id | 4 start_date | 5 planned_end_date | 6 actual_end_date | 7 planned_duration_days | 8 actual_duration_days | 9 is_overdue |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 2 | 2024-10-01 | 2024-10-22 | 2024-10-20 | 21 | 19 | 0 |
| 2 | 2 | 2 | 3 | 2024-10-05 | 2024-10-19 | 2024-10-18 | 14 | 13 | 0 |
| 3 | 3 | 3 | 4 | 2024-11-01 | 2024-11-11 | 2024-11-13 | 10 | 12 | 1 |
| 4 | 4 | 4 | 6 | 2024-11-05 | 2024-12-05 | 2024-12-01 | 30 | 26 | 0 |
| 5 | 5 | 6 | 7 | 2024-11-10 | 2024-11-24 | 2024-11-23 | 14 | 13 | 0 |
| 6 | 6 | 7 | 9 | 2025-11-10 | 2025-11-24 | (NULL) | 14 | (NULL) | 1 |
| 7 | 7 | 9 | 10 | (NULL) | (NULL) | (NULL) | (NULL) | (NULL) | 0 |
| 8 | 8 | 10 | 1 | 2025-10-20 | 2025-10-27 | (NULL) | 7 | (NULL) | 1 |
| 9 | 9 | 1 | 4 | 2025-11-01 | 2025-11-15 | (NULL) | 14 | (NULL) | 1 |
| 10 | 10 | 2 | 1 | (NULL) | (NULL) | (NULL) | (NULL) | (NULL) | 0 |
| 11 | 11 | 1 | 2 | 2026-02-11 | 2026-02-25 | 2026-02-11 | 14 | 0 | 0 |
| 12 | 12 | 1 | 2 | 2026-02-11 | 2026-02-25 | 2026-02-11 | 14 | 0 | 0 |
| 13 | 13 | 1 | 2 | 2026-02-11 | 2026-02-25 | 2026-02-11 | 14 | 0 | 0 |
| 14 | 14 | 1 | 3 | (NULL) | (NULL) | (NULL) | (NULL) | (NULL) | 0 |
| 15 | 15 | 1 | 3 | (NULL) | (NULL) | (NULL) | (NULL) | (NULL) | 0 |
| 16 | 16 | 1 | 3 | (NULL) | (NULL) | (NULL) | (NULL) | (NULL) | 0 |

## ABLAUF

- Validierung der berechneten Leihdauer-Metriken und Identifikation von systemweiten Fristüberschreitungen.
- Relevanz: Fungiert als Steuerungsinstrument zur Überwachung der Prozess-Compliance und liefert die Datengrundlage für die Optimierung der Rücklauflogistik.

vw_dm_loan_fact (16r × 9c) | vw_dm_loan_fact (1r × 1c)

| # | 1 avg_actual_duration_days |
|---|---|
| 1 | 10,38 |

# Teil V: Rollenmodell und Berechtigungen

# Überblick: Rollenmodell

- **Zwei Rollen:** app_member und app_admin
- Umsetzung des Least-Privilege-Prinzips auf Datenbankebene
- Mitglieder sehen personenbezogene Daten ausschließlich über Privacy-Views
- Admins verfügen über vollständige DML-Rechte
- Analyse-Views sind bewusst nur für Admins freigeschaltet

| Bereich | app_member | app_admin |
|---|---|---|
| **Personenbezogene Daten** | Nur über Privacy-Views | Voller Zugriff |
| **Katalogdaten** | Lesen | Lesen + Schreiben |
| **Transaktionsdaten** | Lesen + Schreiben | Lesen + Schreiben |
| **Analyse-Views** | Kein Zugriff | Voller Zugriff |
| **Zweck der Rolle** | Endnutzer der App | Interne Verwaltung und Reporting |

# Privacy-Layer: Schutz sensibler Daten

## SQL STATEMENT

```sql
CREATE OR REPLACE VIEW vw_public_member_profile AS
SELECT
    m.id,
    m.first_name,
    m.last_name,
    m.about_me,
    m.status
FROM member m;

CREATE OR REPLACE VIEW vw_location_search AS
SELECT
    l.id,
    l.city,
    l.postal_code,
    l.country_code,
    l.coordinates
FROM location l;
```

## ABLAUF

- Views projizieren sensible Daten kontrolliert
- **vw_public_member_profile:** nur öffentliche Profildaten
- **vw_location_search:** ohne Straße/Hausnummer
- Mitglieder greifen nie auf personenbezogene Daten zu
- Tests können auf den nachfolgenden Folien zu app_member und app_admin eingesehen werden

# Test app_member: Erlaubte Zugriffe

## SQL STATEMENT

```
SELECT * FROM vw_public_member_profile LIMIT 5;
SELECT * FROM vw_location_search LIMIT 5;
```

vw_public_member_profile (5r × 5c) | vw_location_search (5r × 5c)

| # | 1 id | 2 first_name | 3 last_name | 4 about_me | 5 status |
|---|---|---|---|---|---|
| 1 | 1 | Anna | Müller | Passionate reader of classic literature and philosophy. Al... | active |
| 2 | 2 | Ben | Schmidt | Science fiction enthusiast. Love trading sci-fi and fantasy... | active |
| 3 | 3 | Carla | Schneider | Platform administrator. Here to help with any issues! | active |
| 4 | 4 | David | Fischer | Looking for Spanish and Latin American literature. Open t... | active |
| 5 | 5 | Eva | Weber | Taking a break from reading. Will be back soon! | inactive |

vw_public_member_profile (5r × 5c) | vw_location_search (5r × 5c)

| # | 1 id | 2 city | 3 postal_code | 4 country_code | 5 coordinates |
|---|---|---|---|---|---|
| 1 | 1 | Berlin | 10115 | DE | æ□ □□ □Âõ(\Ï*@Ãõ(\ÞJ@ |
| 2 | 2 | Hamburg | 20095 | DE | æ□ □□ □□□?Æü#@-iÞqŠÆJ@ |
| 3 | 3 | München | 80331 | DE | æ□ □□ D⊲çû)'@EØõôJ□H@ |
| 4 | 4 | Berlin | 10557 | DE | æ□ □□ □4□6<½*@&S□£ÞJ@ |
| 5 | 5 | Köln | 50667 | DE | æ□ □□ Ü×sFÔ□@ xI@ |

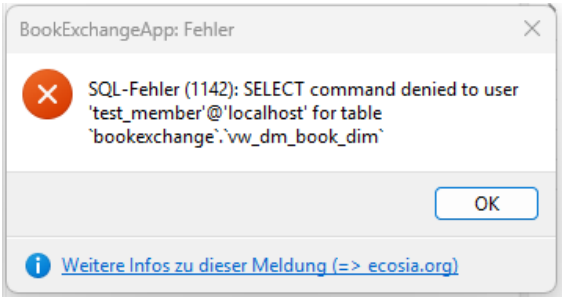
## ABLAUF

- Lesen der Privacy-Views funktioniert wie vorgesehen
- Katalog- und Metadaten können ohne Einschränkungen abgerufen werden
- **Bestätigt:** Member-Rolle besitzt alle für die Anwendung erforderlichen Rechte
- Sensible personenbezogene Daten bleiben geschützt und unsichtbar

# Test app_member: Verbotene Zugriffe auf personenbezogene Daten

## SQL STATEMENT

```
SELECT id, email_address, password_hash FROM member LIMIT 5;
SELECT street, house_number, label FROM location LIMIT 5;
SELECT * FROM vw_dm_book_dim LIMIT 5;
```




## ABLAUF

- Direkter Zugriff auf member wird korrekt blockiert.
- Direkter Zugriff auf location ebenfalls blockiert.
- Zugriff auf Data-Mart-Views ist nicht erlaubt.
- Datenschutz durch Datenbankschicht und Trennung von operativem Zugriff und Reporting bestätigt.

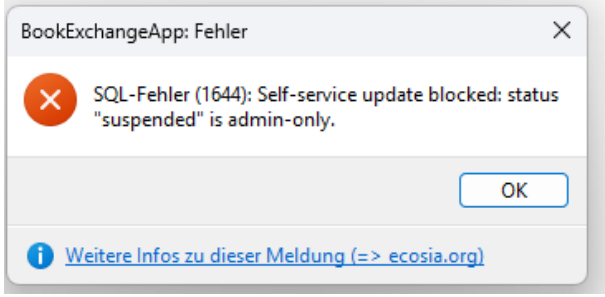
# Test app_member: Profil und Status

## SQL STATEMENT

```
SET @current_member_id = 2;

UPDATE member
SET email_address = 'bennew@example.com',
    about_me      = 'Updated by app_member'
WHERE id = 2;

UPDATE member SET status = 'inactive' WHERE id = 2;

UPDATE member SET status = 'suspended' WHERE id = 2;
```

member (1r × 4c)

| # | 1 id | 2 email_address | 3 about_me | 4 status |
|---|---|---|---|---|
| 1 | 2 | bennew@example.com | Updated by app_member | inactive |



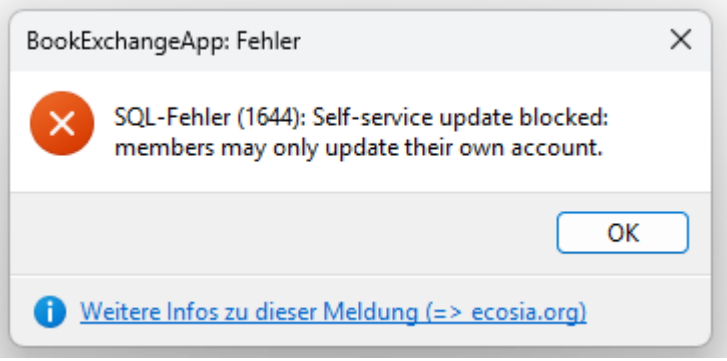

## ABLAUF

- app_member darf eigene Profilfelder aktualisieren (z. B. E-Mail-Adresse, About Me)
- @current_member_id stellt sicher, dass nur der eigene Datensatz geändert wird
- **Erlaubte Statuswerte:** active, inactive, deleted
- **Verboten:** suspended (Admin-only)
- Trigger erzwingt alle Regeln (Row-Ownership + Statusänderungen)

# Test app_member: Schutz vor Fremdänderungen

## SQL STATEMENT

```
SET @current_member_id = 2;

UPDATE member SET email_address = 'hijack@example.com' WHERE id = 3;
```




## ABLAUF

- Members dürfen nur ihren eigenen Datensatz aktualisieren
- **Vergleich:** OLD.id = @current_member_id
- Versuch, fremde IDs zu ändern -> Trigger blockiert
- Umsetzung entspricht Least-Privilege-Prinzip

# Test app_admin: Lesen sensibler Daten

## SQL STATEMENT

```
SELECT id, first_name, last_name, email_address, role FROM member LIMIT 5;
```

member (5r × 5c)

| # | 1 id | 2 first_name | 3 last_name | 4 email_address | 5 role |
|---|---|---|---|---|---|
| 1 | 1 | Anna | Müller | anna.mueller@example.com | member |
| 2 | 2 | Ben | Schmidt | ben.schmidt@example.com | member |
| 3 | 3 | Carla | Schneider | carla.schneider@example.com | admin |
| 4 | 4 | David | Fischer | david.fischer@example.com | member |
| 5 | 5 | Eva | Weber | eva.weber@example.com | member |

## ABLAUF

- Admin erhält vollständigen Lesezugriff auf die Mitgliedertabelle
- Zugriff umfasst auch sensible Attribute wie E-Mail-Adresse
- **Bestätigt:** Admin kann Benutzer verwalten und administrieren
- Zugriff erfolgt wie definiert ohne Einschränkungen

# Test app_admin: Zugriff auf alle Views

## SQL STATEMENT

```
SELECT * FROM vw_public_member_profile LIMIT 5;
SELECT * FROM vw_location_search LIMIT 5;
SELECT * FROM vw_dm_book_dim LIMIT 5;
```

vw_public_member_profile (5r × 5c) | vw_location_search (5r × 5c) | book (5r × 8c)

| # | 1 id | 2 first_name | 3 last_name | 4 about_me | 5 status |
|---|---|---|---|---|---|
| 1 | 1 | Anna | Müller | Passionate reader of classic literature and philosophy. Al... | active |
| 2 | 2 | Ben | Schmidt | Science fiction enthusiast. Love trading sci-fi and fantasy... | active |
| 3 | 3 | Carla | Schneider | Platform administrator. Here to help with any issues! | active |
| 4 | 4 | David | Fischer | Looking for Spanish and Latin American literature. Open t... | active |
| 5 | 5 | Eva | Weber | Taking a break from reading. Will be back soon! | inactive |

vw_public_member_profile (5r × 5c) | vw_location_search (5r × 5c) | book (5r × 8c)

| # | 1 id | 2 city | 3 postal_code | 4 country_code | 5 coordinates |
|---|---|---|---|---|---|
| 1 | 1 | Berlin | 10115 | DE | æ□ □□ □Âõ(\Ï*@Ãõ(\ÞJ@ |
| 2 | 2 | Hamburg | 20095 | DE | æ□ □□ □□□?Æü#@-ÞqŠÆJ@ |
| 3 | 3 | München | 80331 | DE | æ□ □□ D¹çû)'@EØðôJ□H@ |
| 4 | 4 | Berlin | 10557 | DE | æ□ □□ □4□6<½*@&S□£ÞJ@ |
| 5 | 5 | Köln | 50667 | DE | æ□ □□ Ü×sFÔ□@ xI@ |

vw_public_member_profile (5r × 5c) | vw_location_search (5r × 5c) | book (5r × 8c)

| # | 1 book_id | 2 title | 3 subtitle | 4 isbn | 5 publication_year | 6 publisher_name | 7 language_name | 8 primary_genre |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Der Prozess | (NULL) | 9783518188000 | 1925 | Suhrkamp Verlag | Deutsch | Roman |
| 2 | 2 | 1Q84 | Buch 1 & 2 | 9783832161486 | 2010 | Kiepenheuer & Witsch | Deutsch | Roman |
| 3 | 3 | The Handmaids Tale | (NULL) | 9780385490818 | 1985 | Penguin Random House | English | Dystopie |
| 4 | 4 | La Sombra del Viento | (NULL) | 9788408043640 | 2001 | Fischer Verlag | Español | Historischer Roman |
| 5 | 5 | Lamica geniale | (NULL) | 9788866320326 | 2011 | Suhrkamp Verlag | Italiano | Roman |

## ABLAUF

- Admin kann Privacy- und Analyse-Views vollständig auslesen
- Zugriff auf aggregierte Reporting-Daten und Dimensionstabellen bestätigt
- Views liefern vollständige Informationen für Auswertungen und Administration
- Keine Einschränkungen durch Rollen- oder Spaltenfilter

# Test app_admin: Schreiben in Stammdaten

## SQL STATEMENT

```sql
INSERT INTO genre (name, description)
VALUES ('Test-Genre-Admin', 'Temporary admin test entry');

DELETE FROM genre WHERE name = 'Test-Genre-Admin';
```

```sql
INSERT INTO genre (name, description)  VALUES ('Test-Genre-Admin', 'Temporary admin test entry');
DELETE FROM genre WHERE name = 'Test-Genre-Admin';
/* Betroffene Zeilen: 2  Gefundene Zeilen: 0  Warnungen: 0  Dauer von 2 Abfragen: 0,000 Sek. */
```

## ABLAUF

- Admin besitzt volle DML-Berechtigungen auf Stammdaten
- **Test zeigt:** Anlegen und Löschen eines Genres funktioniert fehlerfrei
- **Bestätigt:** Admin kann Katalogdaten pflegen und korrigieren
- Schreibrechte sind korrekt und ausschließlich Admins vorbehalten

# Teil VI: Audit-Trail

# Tabelle admin_audit_log

## SQL STATEMENT

```sql
CREATE TABLE admin_audit_log (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    table_name VARCHAR(80) NOT NULL,
    action ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL,
    record_id BIGINT NOT NULL,
    old_data JSON,
    new_data JSON,
    changed_by VARCHAR(255) NOT NULL,  -- usually USER()
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
  COMMENT='Audit trail for administrative changes';
```

## ABLAUF

- Protokolliert alle administrativen Änderungen an sensiblen Tabellen
- Speichert alte und neue Werte als JSON-Daten
- **Erfasst:** Tabelle, Aktion, Datensatz-ID, Admin-User, Zeitstempel
- Grundlage für Nachvollziehbarkeit und revisionssichere Änderungen

# Trigger: trg_member_audit_update

## SQL STATEMENT

```sql
DELIMITER //
CREATE TRIGGER trg_member_audit_update
BEFORE UPDATE ON member
FOR EACH ROW
BEGIN
    IF USER() LIKE 'test_admin@%' THEN
        INSERT INTO admin_audit_log(
            table_name, action, record_id,
            old_data, new_data, changed_by
        )
        VALUES (
            'member',
            'UPDATE',
            OLD.id,
            JSON_OBJECT(
                'first_name', OLD.first_name,
                'last_name', OLD.last_name,
                'email_address', OLD.email_address,
                'status', OLD.status,
                'role', OLD.role
            ),
            JSON_OBJECT(
                'first_name', NEW.first_name,
                'last_name', NEW.last_name,
                'email_address', NEW.email_address,
                'status', NEW.status,
                'role', NEW.role
            ),
            USER()
        );
    END IF;
END;
//
DELIMITER ;
```

## ABLAUF

- BEFORE UPDATE Trigger auf member
- Protokolliert nur Admin-Aktionen (Überprüfung über USER)
- Speichert vollständige Vorher/Nachher-Werte
- **Erfasst:** Name, E-Mail, Status, Rolle
- **Sicherstellung:** Jede administrative Änderung bleibt nachvollziehbar

# Test: Audit-Trail Validierung

## SQL STATEMENT

```
UPDATE member SET status = 'inactive' WHERE id = 1;

SELECT * FROM admin_audit_log
ORDER BY id DESC LIMIT 1;
```

5 old_data

ıail@example.com", "status": "active", "role": "member"} ...

6 new_data

il@example.com", "status": "inactive", "role": "member"} ...

## ABLAUF

- Testlauf als test_admin
- UPDATE auf member löst Audit-Trigger aus
- **Erwartung:** Neuer Eintrag in admin_audit_log
- old_date enthält Vorher-Werte, new_data die Änderungen
- **Auditorischer Nachweis:** Trigger funktioniert wie vorgesehen

admin_audit_log (1r × 8c)

| # | 1 id | 2 table_name | 3 action | 4 record_id | 5 old_data | 6 new_data | 7 changed_by | 8 changed_at |
|---|---|---|---|---|---|---|---|---|
| 1 | 12 | member | UPDATE | 1 | {"first_name": "Anna", "last_name": "Müller", "email_addr... | {"first_name": "Anna", "last_name": "Müller", "email_addr... | root@localhost | 2026-02-27 20:33:44 |